

SQL Guide and examples (using sakila db)

By Braulio Santa Fe

1. Orden de Ejecución de un Query

1. **FROM / JOIN:** ¿De qué tablas saco los datos y cómo las uno?
2. **WHERE:** ¿Qué filas individuales descarto primero?
3. **GROUP BY:** ¿Cómo agrupo los datos sobrevivientes? (como COUNT).
4. **HAVING:** ¿Qué grupos elimino basados en mis cálculos (SUM, COUNT)?
5. **SELECT:** ¿Qué columnas muestro finalmente en pantalla?
6. **ORDER BY:** ¿Cómo ordeno el resultado final (ASC / DESC)?
7. **LIMIT:** ¿Cuántas filas quiero mostrar como máximo?

2. Checklist Mental

1. ¿Funciones con columnas? **GROUP BY.**
2. ¿Filtrar SUM o COUNT? **HAVING.**
3. ¿Campos vacíos? **IS NULL.**
4. ¿Muchos a muchos? **Tabla Puente.**

3. Tipos de Relaciones

1. **1:N:** Un A para varios B. Ejemplo: País y Ciudades.
2. **1:1:** Relación exacta. Ejemplo: Cliente y Expediente.
3. **M:N:** Requiere tabla puente. Ejemplo: Películas y Actores.

4. Tabla Intermedia

- **Ruta:** A → Puente → B.
- **Solución:** Evita duplicidad.
- **Ventaja:** Escalabilidad.

5. Restricciones (Constraints)

- **PK:** Identificador único.
- **FK:** Conecta tablas.
- **NOT NULL:** Obligatorio.
- **UNIQUE:** Sin repetir.

6. Tipos de Datos

INT	Enteros.
VARCHAR	Texto.
DATE	Fechas.
DECIMAL	Precisión.

7. Funciones Comunes

- **Agregación:** SUM, AVG, COUNT, MAX, MIN.
- **Texto:** CONCAT, UPPER, SUBSTRING.
- **Lógica:** CASE WHEN ... THEN ... END.

8. Tipos de JOINS

INNER	Solo coincidencias.
LEFT	Todo A + coincidencia B.
RIGHT	Todo B + coincidencia A.

30 Challenges in SQL using Sakila Database

- 1. Lista todos los actores con el apellido 'Gibson'.

```
SELECT first_name, last_name from actor
WHERE last_name = "GIBSON";
```

- 2. Encuentra todos los clientes cuyo nombre empieza con 'A'.

```
SELECT first_name, last_name from customer
where first_name LIKE 'A%';
```

- 3. Muestra los títulos de las películas con una duración mayor a 180 minutos.

```
SELECT title from film
WHERE LENGTH > 180;
```

- 4. Encuentra los pagos realizados entre el 25 de mayo y el 16 de junio de 2005.

```
select payment_id, payment_date from payment
where payment_date between '2005-05-25' and '2005-06-16';
```

- 5. ¿Cuántas películas hay en la base de datos?

```
select count(distinct title) from film;
```

- 6. Lista los nombres de todas las categorías de películas.

```
SELECT distinct name from category;
```

– 7. Obtén el nombre y apellido de los actores ordenados alfabéticamente por apellido.

```
select concat(first_name, '␣', last_name) from actor
order by last_name asc;
```

– 8. Encuentra todas las películas con clasificación 'PG-13'.

```
select title from film
WHERE rating = 'PG-13';
```

– 9. ¿Cuántas películas hay por cada clasificación (rating)?

```
select count(*) from film
group by rating;
```

– 10. Encuentra las 5 películas más caras de alquilar (rental_rate).

```
select title, rental_rate from film
order by rental_rate desc
limit 5;
```

– 11. ¿Qué actores han aparecido en la película 'ACADEMY DINOSAUR'?

```
select actor.first_name, actor.last_name
from actor
join film_actor on actor.actor_id = film_actor.actor_id
join film on film.film_id = film_actor.film_id
where film.title = 'ACADEMY␣DINOSAUR';
```

otra forma

```
SELECT actor.first_name, actor.last_name
FROM actor
JOIN film_actor USING (actor_id)
JOIN film USING (film_id)
WHERE film.title = 'ACADEMY_DINOSAUR';
```

– 12. Encuentra todos los clientes que viven en 'California'.

```
select c.first_name, c.last_name
from customer c
join address a using(address_id)
where a.district = 'California';
```

otra forma

```
select c.first_name, c.last_name
from customer c
join address a on c.address_id = a.address_id
where a.district = 'California';
```

– 13. Muestra el inventario de la tienda número 1.

```
select *
from inventory i
join store s using(store_id)
where store_id = 1;
```

otra forma

```
select *  
from inventory i  
join store s on i.store_id = s.store_id  
where s.store_id = 1;
```

– 14. Encuentra todas las películas donde el actor 'NICK WAHLBERG' aparece.

```
select f.title  
from film f  
join film_actor fa on f.film_id = fa.film_id  
join actor a on fa.actor_id = a.actor_id  
where a.first_name = 'NICK' and a.last_name = 'WAHLBERG';
```

TIP para evitar confusion Con CONCAT: El motor no puede usar el índice. Para saber si el resultado del CONCAT es igual a 'NICK WAHLBERG', SQL está obligado a: Leer cada fila de la tabla una por una.

Unir el nombre y el apellido de esa fila.

Comparar el resultado con 'NICK WAHLBERG'.

Repetir esto para miles de filas.

– 15. ¿Cuál es la duración promedio de las películas por categoría?

```
select c.name, avg(f.length) from film f  
join film_category fc on fc.film_id = f.film_id  
join category c on c.category_id = fc.category_id  
group by c.name;
```

otra forma

```
select name, avg(length) from film
join film_category using(film_id)
join category using (category_id)
group by name;
```

Recordatorio: Diferencias entre INNER JOIN, LEFT JOIN y RIGHT JOIN

1. INNER JOIN (Solo lo que coincide en ambos lados)

Aquí le dices a la base de datos: “Quiero ver las películas, pero solo si tienen copias físicas en el inventario”.

```
SELECT f.title, i.inventory_id
FROM film f
INNER JOIN inventory i ON f.film_id = i.film_id;
```

Resultado: Si una película no tiene copias en el inventario, desaparece por completo de la lista. Solo verás las películas que la gente realmente puede ir a rentar hoy.

2. LEFT JOIN (Todo lo de la izquierda, coincida o no)

Aquí pones a la película a la izquierda y dices: “Quiero ver todas las películas de mi catálogo. Si tienen copias, muéstrame el ID de la copia; si no tienen, muestra un vacío (NULL)”.

```
SELECT f.title, i.inventory_id
FROM film f
LEFT JOIN inventory i ON f.film_id = i.film_id;
```

3. RIGHT JOIN (Todo lo de la derecha, coincida o no)

Aquí invertimos la jerarquía. La tabla de la derecha es `inventory`. Le dices a la base de datos: “Quiero ver todas las copias físicas que existen en mis pasillos. Si esa copia corresponde a una película de mi catálogo, ponle su título; si no, déjalo en NULL”.

```
SELECT f.title, i.inventory_id
FROM film f
RIGHT JOIN inventory i ON f.film_id = i.film_id;
```

– 16. ¿Qué clientes no han hecho ningún alquiler?

```
select c.first_name from customer c
left join rental r on c.customer_id = r.customer_id
where r.rental_id is null;
```

– 17. Encuentra la cantidad total de dinero recaudado por cada tienda.

```
select s.store_id, sum(p.amount) from store s
join customer c on c.store_id = s.store_id
join payment p on p.customer_id = c.customer_id
group by s.store_id;
```

— 18. Lista las películas que tienen la palabra 'Action' en su categoría.

```
select title, name from film
join film_category using(film_id)
join category using(category_id)
where name = 'Action';
```

TIP para evitar confusion Where vs Having

WHERE: Filtra filas individuales **antes** de agrupar.

- No admite funciones de agregación (SUM, COUNT, etc.).
- Ejemplo: WHERE tienda = 1

HAVING: Filtra grupos **después** de agrupar.

- Sí admite funciones de agregación.
- Ejemplo: HAVING COUNT(*) >20

– 19. Encuentra los nombres y numero de peliculasde los actores que han actuado en más de 20 películas.

```
select a.first_name, count(fa.film_id) from actor a
join film_actor fa on fa.actor_id = a.actor_id
group by a.first_name
having count(fa.film_id) > 20;
```

– 20. Muestra el título de la película y el nombre del idioma en el que está.

```
select f.title, l.name from film f
join language l on f.language_id = l.language_id;
```

– 21. ¿Qué ciudades no tienen tiendas asignadas?

1. LEFT JOIN + IS NULL

Enfoque: “Intenta conectar todo y quédate con lo que falló”.

Mecánica: Une las tres tablas (`city` ->`address` ->`store`) creando una tabla virtual grande. Luego, el `WHERE` filtra las filas donde la parte de `store` resultó vacía (`NULL`).

Cuándo usar: Es muy intuitivo cuando quieres visualizar cómo se relacionan las tablas a través de sus claves foráneas.

Listing 1: Enfoque LEFT JOIN

```
SELECT c.city
FROM city c
LEFT JOIN address a ON c.city_id = a.city_id
LEFT JOIN store s ON a.address_id = s.address_id
WHERE s.store_id IS NULL;
```

2. NOT EXISTS

Enfoque: “Por cada fila de A, comprueba si existe una relación en B”.

Mecánica: Es una subconsulta correlacionada. Para cada ciudad, pregunta: “¿Hay alguna tienda asociada?”. Si la respuesta es “no”, la ciudad se incluye.

Cuándo usar: Es el estándar profesional para verificar existencia. Suele ser más eficiente en motores de bases de datos porque el motor puede detener la búsqueda apenas encuentre la primera coincidencia.

Listing 2: Enfoque NOT EXISTS

```
SELECT c.city
FROM city c
WHERE NOT EXISTS (
    SELECT 1
    FROM address a
    JOIN store s ON a.address_id = s.address_id
    WHERE a.city_id = c.city_id
);
```

– 22. Encuentra las películas que fueron alquiladas en junio de 2005.

```
select distinct f.title from film f
join inventory i on f.film_id = i.film_id
join rental r on i.inventory_id = r.inventory_id
where rental_date >= '2005-06-01' and rental_date < '2005-07-01';
```

– 23. ¿Cuál es el actor que ha aparecido en más películas?

```
select a.actor_id, a.first_name, count(fa.film_id) from actor a
join film_actor fa on a.actor_id = fa.actor_id
group by a.actor_id, a.first_name, a.last_name
order by count(fa.film_id) desc
limit 1;
```

– 24. Muestra los nombres de los clientes y cuántas películas han alquilado.

```
select c.first_name, c.last_name , count(c.customer_id) from customer c
join rental r on c.customer_id = r.customer_id
group by c.customer_id, c.first_name, c.last_name;
```

– 25. Encuentra el cliente que más dinero ha gastado.

```
select c.first_name, c.last_name, sum(p.amount) as total_gastado from customer c
join payment p on p.customer_id = c.customer_id
group by c.customer_id, c.first_name, c.last_name
order by total_gastado desc
limit 1;
```

– 26. Lista todas las películas que incluyen el actor 'PENELOPE GUINESS' y 'LUCILLE TRACY'.

```
select f.title as titulo_film from film f
join film_actor fa on f.film_id = fa.film_id
join actor a on a.actor_id = fa.actor_id
where a.first_name = 'PENELOPE' and a.last_name = 'GUINESS'

intersect

select f.title as titulo_film from film f
join film_actor fa on f.film_id = fa.film_id
join actor a on a.actor_id = fa.actor_id
where a.first_name = 'LUCILLE' and a.last_name = 'TRACY';
```

– 27. Encuentra los nombres de los actores que han actuado en películas de terror (Horror).

```
select distinct a.first_name, a.last_name from actor a
join film_actor fa on fa.actor_id = a.actor_id
join film_category fc on fc.film_id = fa.film_id
join category c on c.category_id = fc.category_id
where c.name = 'Horror';
```

– 28. ¿Qué días se han realizado más de 100 alquileres?

```
select date(rental_date) as fecha, count(rental_id) as numeroventas from rental
group by fecha
having numeroventas > 100;
```

– 29. Muestra el nombre del cliente y el nombre del empleado (staff) que procesó su último alquiler.

```
select concat(c.first_name,'␣', c.last_name) as cliente, concat(s.first_name,'␣', s.
    last_name) as staff from customer c
join rental r on r.customer_id = c.customer_id
join staff s on r.staff_id = s.staff_id
where r.rental_id in (
    select max(rental_id) from rental
    group by customer_id
);
```

– 30. Clasifica las películas en tres grupos según su duración: Corta (<60), Media (60-120), Larga (>120).

```
select title, length as duracion,
    case
        when length < 60 then 'corta'
        WHEN length between 60 and 120 then 'larga'
        else 'large'
    end as clasificacion_duration
from film
order by duracion;
```